

Exercise set 4

Regression Trees

Task 1

- a) When we fit a regression tree, we sequentially divide the predictor space by finding the variable X_j and the cutpoint s that reduces the RSS as much as possible. Predict the natural logarithm of **Salary** using **Hits** and **Years** in the **Hitters** data. The code below is doing the first split in the regression tree. Explain what the code does. Use the help functions in R if necessary.

```
library(ISLR)
RSS=function(s,x1,y)
{
  yR1=mean(y[x1<s])
  yR2=mean(y[x1>=s])
  rss=sum((y[x1<s]-yR1)^2)+sum((y[x1>=s]-yR2)^2)
  return(rss)
}
Hitters=Hitters[!is.na(Hitters$Salary),]
Hitters$logSalary=log(Hitters$Salary)
optimise(RSS,lower=0,upper=250,x1=Hitters$Hits,y=Hitters$logSalary)

## $minimum
## [1] 117.5044
##
## $objective
## [1] 160.9715

optimise(RSS,lower=0,upper=50,x1=Hitters$Years,y=Hitters$logSalary)

## $minimum
## [1] 4.508575
##
## $objective
## [1] 115.0585
```

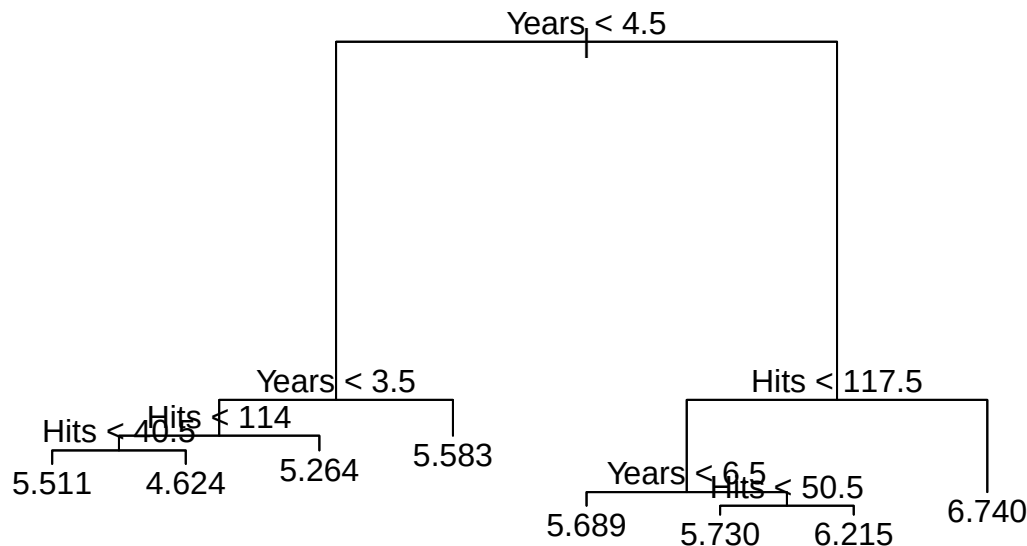
Solution

For a given variable, x_1 , and a cutpoint, the `RSS`-function computes the residual-sum-of-squares. $yR1$ is the average of the y 's in the region defined by $x_1 < s$ and $yR2$ the average in the region defined by $x_1 \geq s$. `Hitters=Hitters[!is.na(Hitters$Salary),]` removes all observations in `Hitters` with missing observations on the salary variable. The two rows with `optimise` is finding the cutpoints that minimizes RSS in the `Hits`- and `Years`-dimension, respectively. The minimum RSS is smallest in the `Years`-dimension. `Years` is therefore chosen as the first variable to split by.

- b) Use the `tree`-function in the `tree`-package to fit and plot a tree for `logSalary` and the variables `Hits` and `Years` only.

```
library(tree)
tree1=tree(logSalary~Hits+Years,data=Hitters)
```

```
plot(tree1)
text(tree1,pretty=0)
```



Can you see where your computations in a) occur in the tree?

Solution The first split was computed to be for $\text{Years} < 4.5$. This is the split visible on the root (first split) of the tree.

c) Predict the salary of a new observation with $\text{Hits}=125$ and $\text{Years}=2.5$.

Solution

$\text{Years} < 4.5$? True, go to the left. $\text{Years} < 3.5$? True, go to the left. $\text{Hits} < 114$? False, go to the right. Prediction=5.264.

d) Fit a tree for $\log\text{Salary}$ with all predictors; call this `tree2`. Compute the RSS for `tree1` and `tree2`. Also, use the summary function to see if you can find RSS there.

Solution

```
tree2=tree(logSalary~.-Salary,data=Hitters)
summary(tree1)
```

```
##
## Regression tree:
## tree(formula = logSalary ~ Hits + Years, data = Hitters)
## Number of terminal nodes: 8
## Residual mean deviance: 0.2708 = 69.06 / 255
## Distribution of residuals:
##   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -2.2400 -0.2980 -0.0365  0.0000  0.3233  2.1520
```

```
summary(tree2)
```

```
##
## Regression tree:
## tree(formula = logSalary ~ . - Salary, data = Hitters)
## Variables actually used in tree construction:
## [1] "CAtBat" "CHits" "AtBat" "CRuns" "Hits" "Walks" "CRBI"
## Number of terminal nodes: 9
## Residual mean deviance: 0.1694 = 43.03 / 254
```

```
## Distribution of residuals:
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -1.7080 -0.2213  0.0353  0.0000  0.2303  1.7020
```

```
rss1=sum(residuals(tree1)^2)
rss2=sum(residuals(tree2)^2)
rss1
```

```
## [1] 69.06105
```

```
rss2
```

```
## [1] 43.03165
```

The number in the denominator of the deviation computation in the summary function is the RSS.

Bagged trees and random forest

Task 2

Bagging is based on the idea of reducing prediction variance by averaging over predictions for many bootstrapped samples.

- a) Use a cross-validation approach to compute the `testMSE` for the `tree2` in Task 1. We are going to use this as a benchmark to compare with bagging, random forest and boosting.

Solution

```
set.seed(123)
n=nrow(Hitters)
ind=sample(1:n,size=floor(n/2))
train=Hitters[ind,]
test=Hitters[-ind,]
tree2=tree(logSalary~.-Salary,data=train)
pred2=predict(tree2,newdata=test)
mse2=mean((test$logSalary-pred2)^2)
mse2
```

```
## [1] 0.428329
```

- b) Use bagging of `tree2` in Task 1 to see if you can improve the predictions. Use the `randomForest` package. Use the variable importance plot, `varImpPlot` and interpret the results. What is the unit of the x-axis of the plot. Use the help-function to the `importance`-function to find this out.

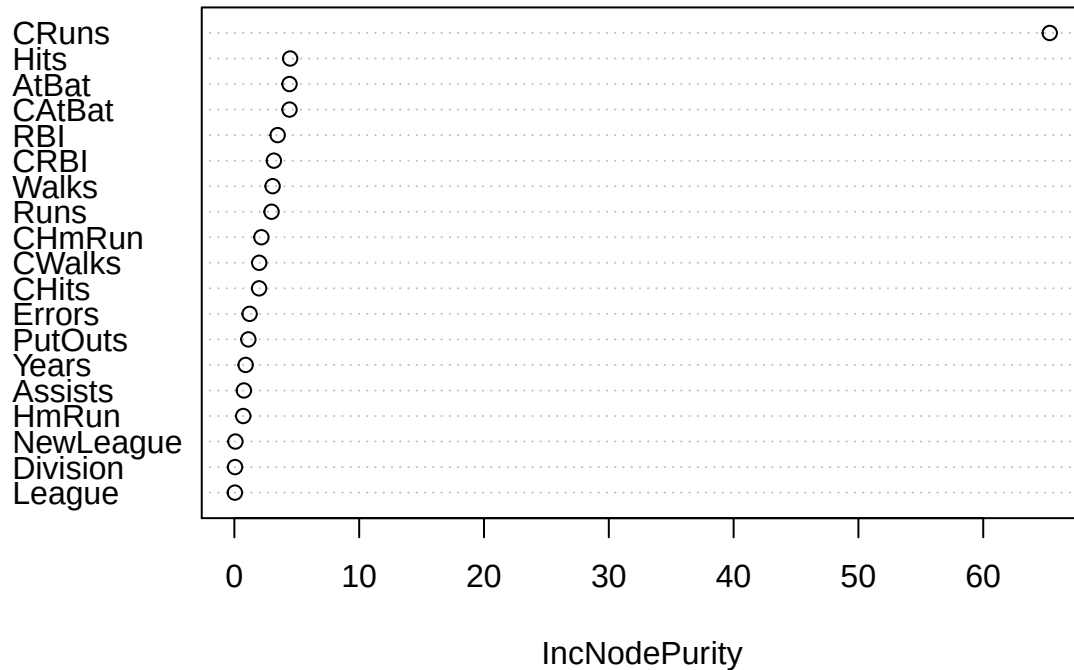
Solution

```
library(randomForest)
bagging=randomForest(logSalary~.-Salary,data=train,mtry=19)
predbag=predict(bagging,newdata=test)
msebag=mean((test$logSalary-predbag)^2)
msebag
```

```
## [1] 0.3015121
```

```
varImpPlot(bagging)
```

bagging



The CRuns variable is by far the most powerful predictor. The number on the x-axis, 65 for CRuns, is the average over all trees of the decrease in RSS due to the predictor.

- c) Can you improve the predictions with random forest? You have to use the `randomForest`-function in a modified way compared to in b). Random forest is an extension of bagging. In what sense? Why are random forests potentially better predictors?

Solution

```
library(randomForest)
rf=randomForest(logSalary~.-Salary,data=train,mtry=3)
predrf=predict(rf,newdata=test)
msef=mean((test$logSalary-predrf)^2)
msef
```

Here only three predictors were sampled for each tree. This improved predictor is, thus, an average of predictions from relatively small trees.

Boosting

In boosting, predictions are also averaged, just like in bagging and random forest. However, the dataset is updated sequentially so that residuals from the previous iteration is used as data in the next.

Task 3

Again, you shall predict `logSalary` in the Hitters data. Use the `gbm`-function in the `gbm`-package to predict with a boosted tree. Compute `testMSE`.

Solution

```
## USING gbm
library(gbm)
```

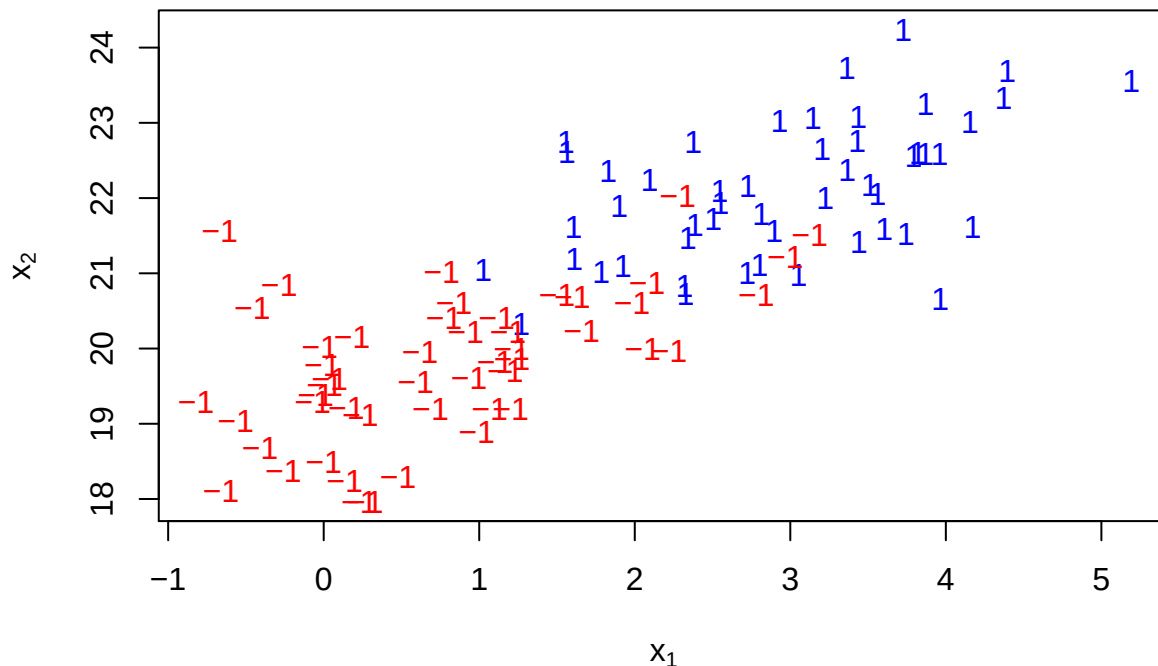
```
## Loaded gbm 2.1.8
boost=gbm(logSalary~.-Salary,data=train,distribution
          ="gaussian",n.trees=1000,interaction.depth=4,
          shrinkage=0.01)
predgbm=predict(boost,newdata=test,n.trees=1000)
msegbm=mean((test$logSalary-predgbm)^2)
msegbm

## [1] 0.2666642
```

Support vector machines

Task 4

- a) In the plot below where the classes “1” and “-1” are denoted, is it possible to find a maximal marginal classifier?



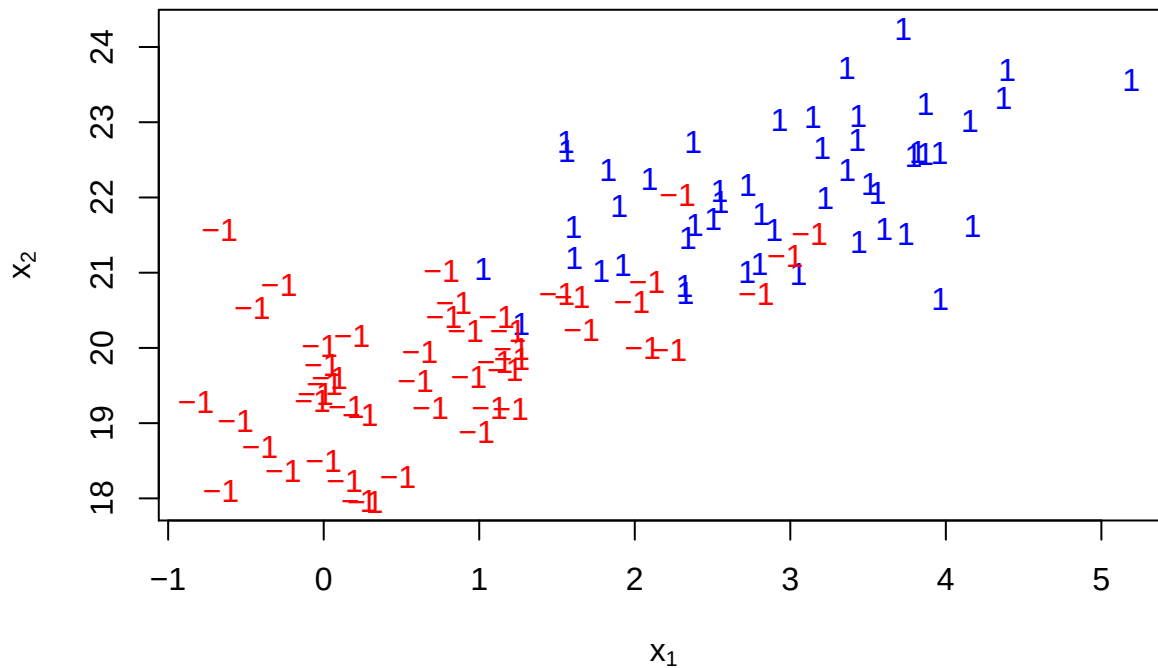
Solution

No, it is not possible to find a line in the (x_1, x_2) -plane which completely separates the “1” and “-1”-observations.

- b) The dataset in a) can be downloaded as the file “xy.csv” in Canvas. Download it, read it into R and recreate the plot in a).

Solution

```
xy=read.csv("xy.csv") # Make sure that your current working
                      # directory is where the file is located
x=xy[,1:2]
y=xy[,3]
plot(x,type="n",xlab=expression(x[1]),ylab=expression(x[2]))
text(x[y==1,],"1",col="blue")
text(x[y== -1,],"-1",col="red")
```

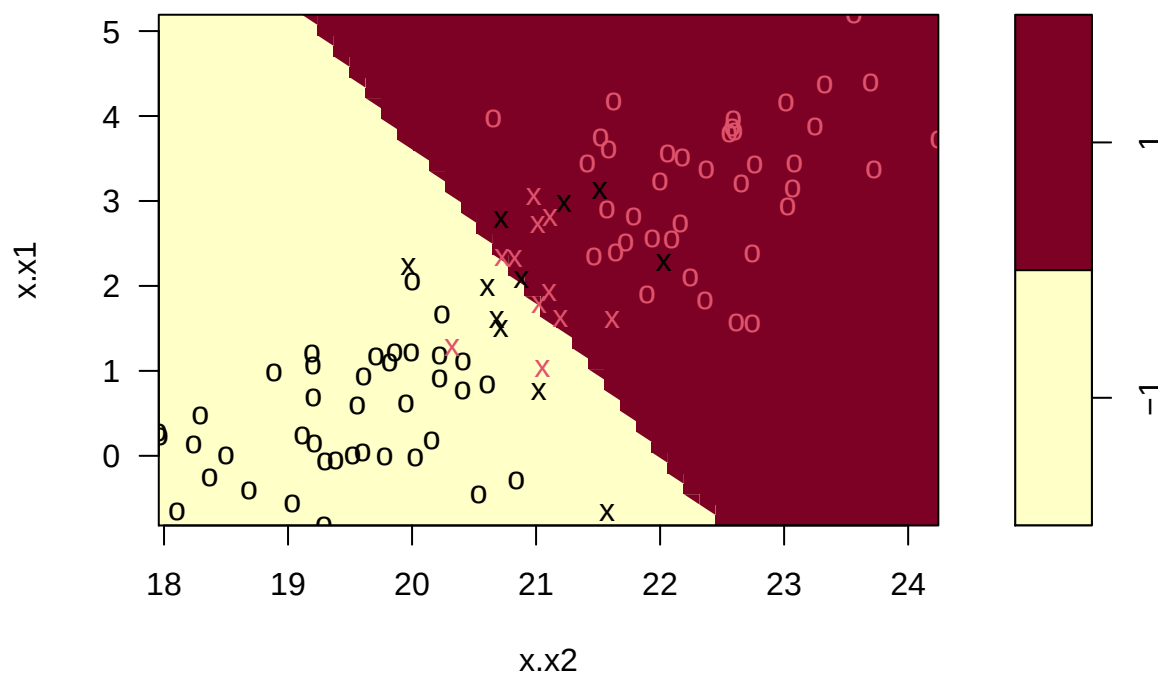


c) Use the support vector classifier to classify the dataset. See lecture13.pdf for how to do this.

Solution

```
library(e1071)
dat=data.frame(x=x,y=as.factor(y))
svmfit=svm(y~.,data=dat,kernel="linear",cost=10,scale=FALSE)
plot(svmfit,dat)
```

SVM classification plot



d) What does it mean to set the tuning parameter C equal to 10?

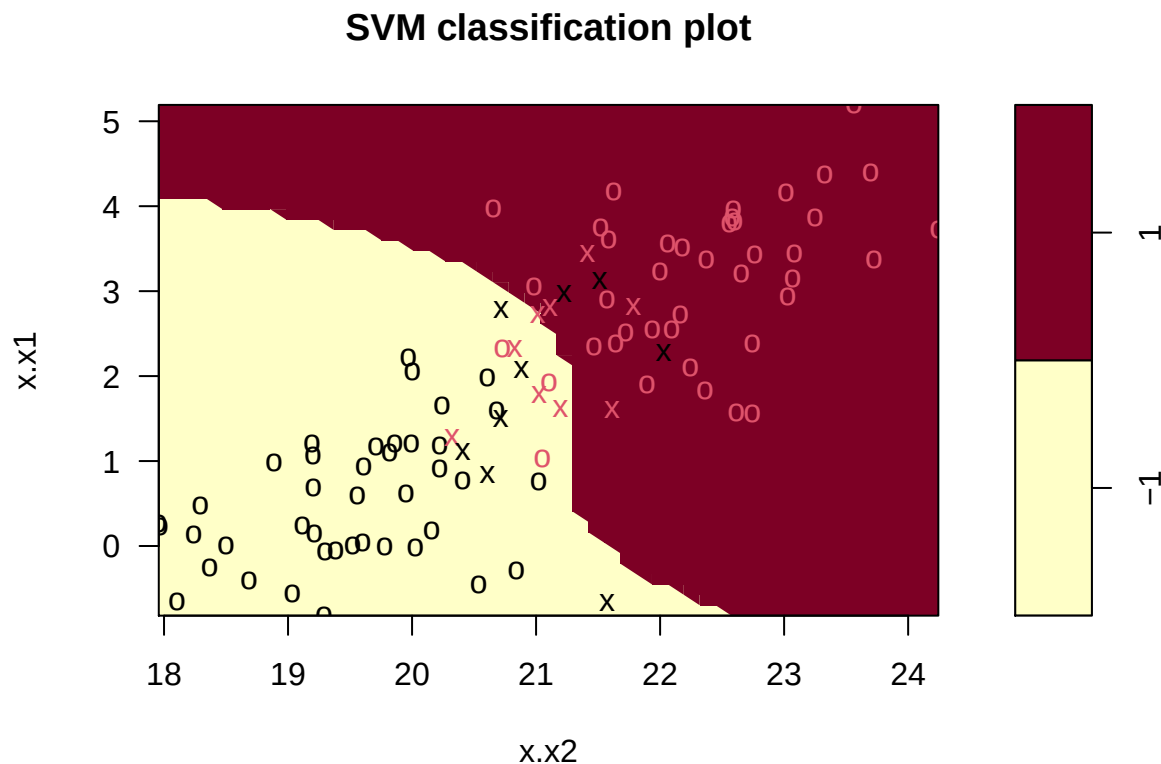
Solution

It means that we allow at most 10 observations to be located on the wrong side of the separating hyperplane.

- e) Try a support vector machine with a polynomial kernel and check if it is able to separate the training data better?

Solution

```
svmfit=svm(y~.,data=dat,kernel="polynomial",cost=10,scale=FALSE)
plot(svmfit,dat)
```



It is better in the training data but as always, a more flexible model is more prone to overfitting. It is not obvious that it is a better classifier in test data.

Principal component analysis (PCA)

Task 5

- a) Use principal component analysis to analyze the `advertising` data in "advertising.csv", which you will find in Canvas. Find the principal first two principal components for the variables TV, radio and newspaper. Interpret them by looking at the loadings.

Solution

```
advertising=read.csv("advertising.csv")[,c("TV","radio","newspaper")]
pca=prcomp(advertising)
summary(pca)
```

```
## Importance of components:
##              PC1      PC2      PC3
## Standard deviation  85.8679 22.71732 13.27840
## Proportion of Variance 0.9142 0.06398 0.02186
```

```
## Cumulative Proportion    0.9142    0.97814    1.00000
```

```
pca$rotation
```

```
##          PC1          PC2          PC3
## TV      -0.99982947 -0.01807077 -0.003805766
## radio   -0.01001377  0.35736234  0.933912137
## newspaper -0.01551648  0.93379098 -0.357482357
```

PC1, accounting for 91% of the variation in the three variables is mainly determined by **TV**. A market with a small value on PC1 implies that the market has a large TV advertising budget. PC2, accounting for 6% of the variation, is mainly determined by **newspaper**. An market with a large value on PC2 has a large weighted average of newspaper and radio advertising budgets. We capture 97% of the variation with two variables instead of the original three.

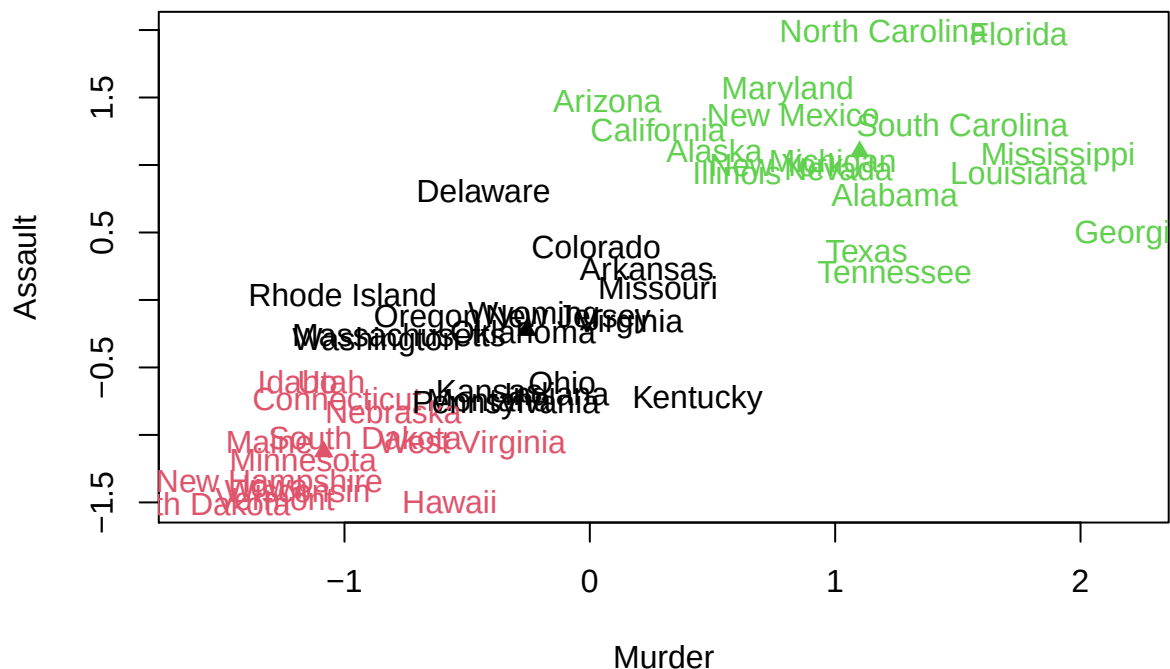
Cluster analysis

Task 6

- a) Use K-means clustering to cluster the states in the USArrest data in three different groups based on the variables **Murder** and **Assault**.

Solution

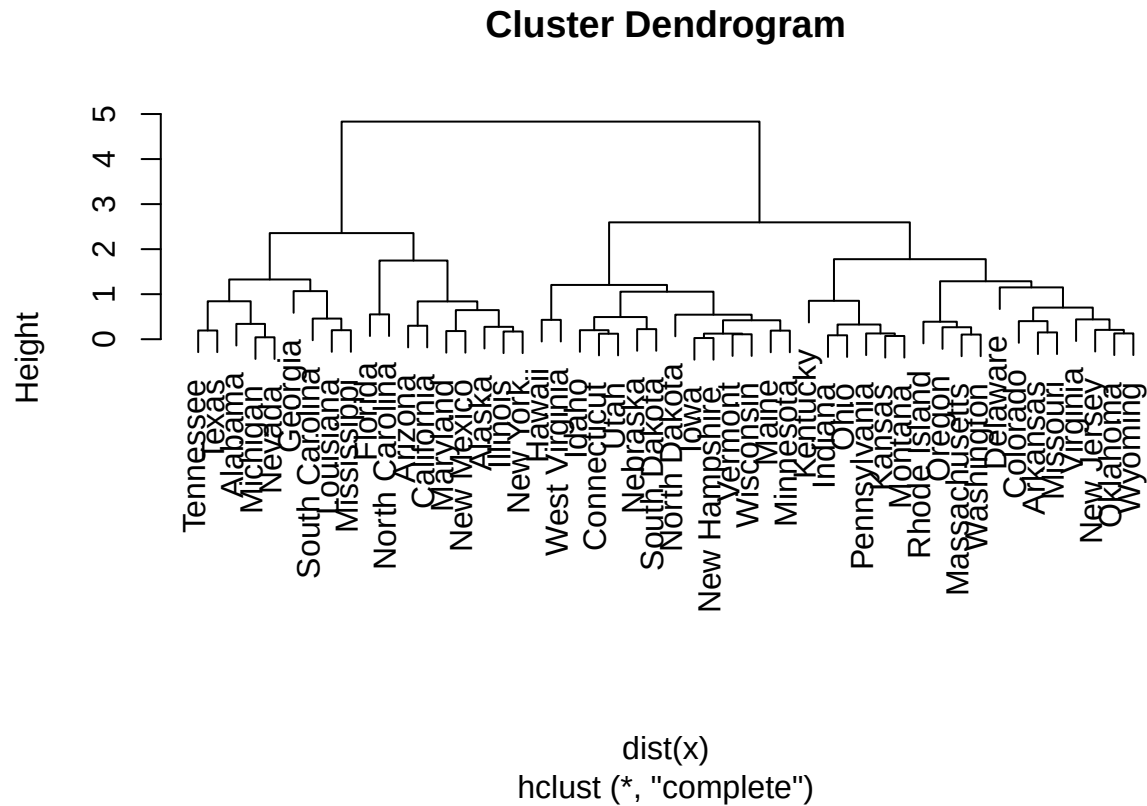
```
x=USArrests[,1:2]
x[,1]=(x[,1]-mean(x[,1]))/sd(x[,1])
x[,2]=(x[,2]-mean(x[,2]))/sd(x[,2])
km3=kmeans(x,3)
plot(x,type="n")
points(km3$centers,pch=17,col=1:3)
text(x,rownames(x),col=km3$cluster)
```



- b) Use hierarchical clustering.

Solution


```
hc11=hclust(dist(x),method="complete")
plot(hc11)
```



If we cut the tree so that we obtain three categories, around the height of 2.5, we see some differences compared to the K-means clustering; the clusters are not the same. Try using another linkage function to see if you obtain a more similar results. Also, try to use all 4 variables rather than just the two used here.